

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	KEERTHANA.S	Reg. No.:	192471047
Date:	18.08.2026	Duration:	60 minutes

Code of Conduct

I KEERTHANA S (192471047) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Keerthana S

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

1. Requirement and Pipeline Analysis

1.1 Project Context

The Smart AI-Based Urban Street Lighting Management System is a multi-service application: a Node.js/Express backend API, a Python/FastAPI AI engine (fault detection, adaptive brightness, predictive maintenance), a React monitoring dashboard, a PostgreSQL database, and an MQTT broker for IoT telemetry. Because these services are updated independently and directly affect public infrastructure and safety, changes must be integrated, tested, and released frequently, reliably, and with strong safety nets - which is precisely what a CI/CD pipeline provides.

1.2 CI/CD Needs Identified

- Automatically build and validate every service (backend, ai-engine, frontend) on each commit, since they have different runtimes and cannot be validated with a single generic build step.
- Run automated unit and integration tests on every push/PR so regressions in fault-detection logic or brightness control are caught before merge, not after deployment to real streetlights.
- Enforce code-quality and security gates (linting, static analysis, dependency/container scanning) before code can reach shared branches.
- Automate Docker image building and versioned packaging so every release is reproducible and traceable back to a specific commit.
- Support progressive deployment across Dev → Staging → Production environments, since a defect in AI brightness logic reaching production directly affects public lighting.
- Provide fast rollback and alerting, given the system controls physical infrastructure and cannot tolerate long outages or unnoticed failures.
- Keep secrets (database credentials, JWT keys, registry tokens) out of source control while still being usable by automated pipeline steps.

1.3 Pipeline Objectives

Objective	Description
Fast feedback	Developers learn within minutes whether a change breaks a build or test, not after manual QA.
Consistent builds	Every service is built inside identical containerized environments regardless of the developer's machine.
Quality gates	No code merges to develop/main without passing lint, tests, and security scans.
Traceable releases	Every deployed artifact is tagged with its Git commit SHA and can be traced back to exact source code.
Safe, staged rollout	Changes progress through Dev and Staging before reaching Production, reducing risk to live streetlights.
Quick recovery	A failed production deployment can be rolled back automatically or with a single command.

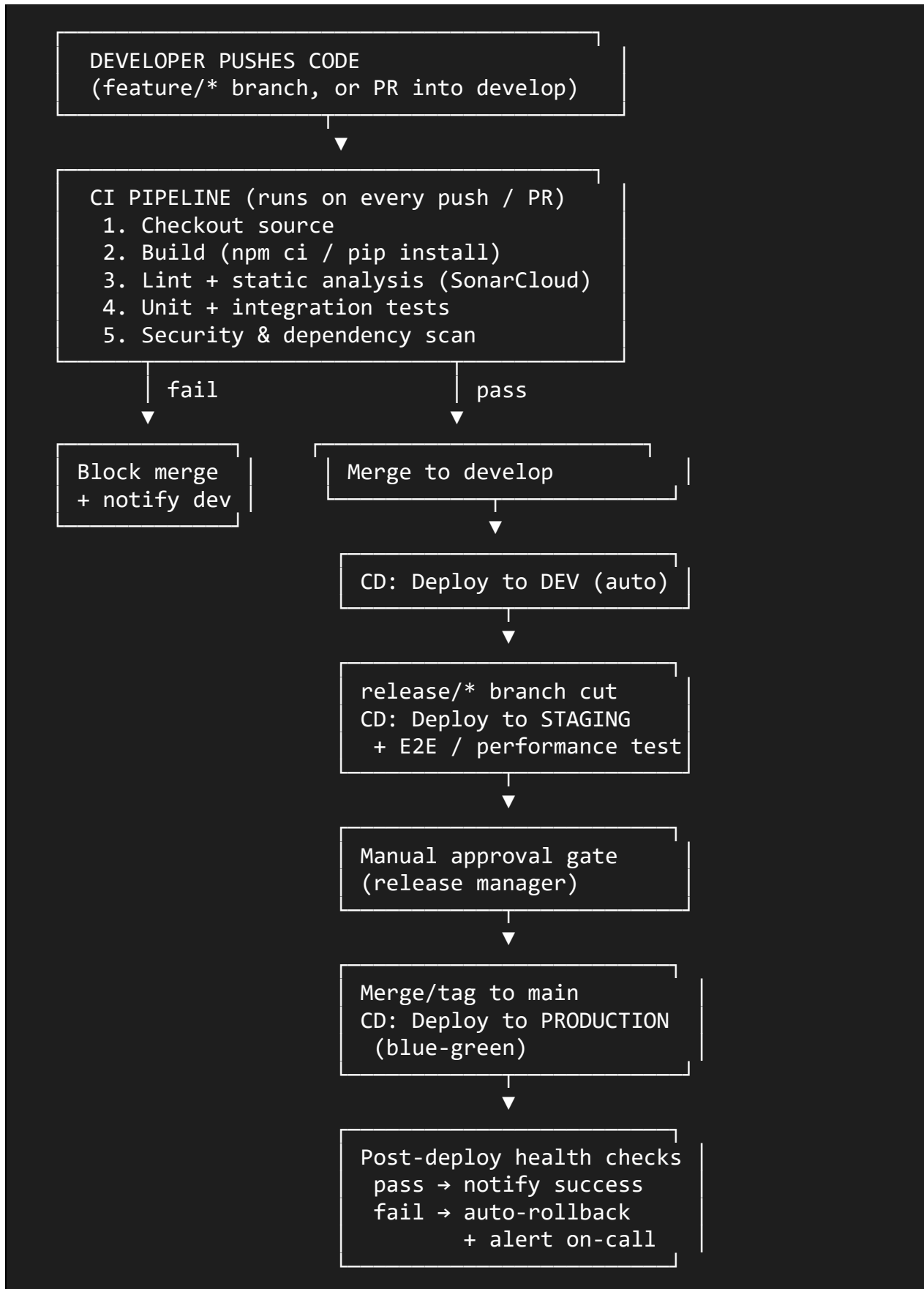
2. Pipeline Architecture and Workflow

The pipeline is implemented as a multi-stage GitHub Actions workflow, triggered on pushes and pull requests, that mirrors the project's Gitflow branching model (main / develop / feature / release / hotfix).

2.1 Pipeline Stages Overview

#	Stage	Purpose
1	Source Code Management (Trigger)	Pipeline triggers on push to feature/*, pull requests into develop, and merges into release/*/main.
2	Build	Install dependencies and compile/build each service (npm ci + build for backend/frontend, pip install for ai-engine).
3	Static Code Analysis & Linting	Run ESLint/Prettier (Node), flake8/black (Python), and SonarCloud static analysis; enforce quality gate.
4	Automated Testing	Run unit tests (Jest, pytest), integration tests, and a Docker-Compose-based end-to-end smoke test.
5	Security Scanning	Dependency scans (npm audit, pip-audit), container image scanning (Trivy), and secret scanning (Gitleaks).
6	Packaging	Build Docker images per service, tag with git SHA + semantic version, push to a container registry (GHCR).
7	Deploy to Dev	Auto-deploy on every merge to develop for continuous integration validation.
8	Deploy to Staging	Auto-deploy on release/* branch creation; run E2E and performance tests against a production-like environment.
9	Manual Approval Gate	A designated release manager approves promotion of the staging build to production.
10	Deploy to Production	Blue-green (or rolling) deployment on merge/tag to main, with automated health checks.
11	Post-Deployment Monitoring	Health checks and metrics are watched for a defined window; automatic rollback triggers on failure.
12	Notifications	Slack/Teams and email notifications fire at each critical stage (build failure, deploy success/failure, rollback).

2.2 Pipeline Diagram



2.3 Workflow Explanation: Commit to Deployment

- A developer pushes to a feature/* branch or opens a pull request into develop; this triggers the CI portion of the pipeline (stages 1-5).
- If build, lint, tests, or security scans fail, the pipeline stops, the PR is blocked from merging, and the developer is notified with the failure details.
- Once CI passes and the PR is reviewed and merged into develop, the pipeline automatically builds and tags Docker images and deploys them to the Dev environment for integration validation.
- When the team is ready to prepare a release, a release/* branch is cut from develop; this triggers deployment to Staging, where E2E and performance tests run against a production-like stack.
- A release manager reviews the staging results and manually approves promotion; merging the release branch into main triggers the Production deployment using a blue-green strategy.
- After production deployment, automated health checks monitor the new version for a defined window; if checks fail, the pipeline automatically rolls back to the previous stable image and alerts the team.
- Notifications are sent at each key transition (CI failure, successful Dev/Staging/Production deploy, rollback) via Slack and email so the team always has visibility into pipeline state.

3. Tools and Technology Selection

Stage	Tool(s)	Justification
Source Code Management	Git + GitHub	Already used for the project's Gitflow branching; GitHub natively integrates with GitHub Actions and PR-based quality gates.
CI/CD Orchestration	GitHub Actions	No separate CI server to maintain, free tier is sufficient for this project's scale, and workflows are defined as versioned YAML alongside the code.
Build - Backend/Frontend	Node.js + npm	Matches the existing Node/React stack; npm ci provides fast, reproducible installs from the lockfile.
Build - AI Engine	Python + pip / venv	Matches the existing FastAPI/ML stack; isolates Python dependencies per service.
Linting/Formatting	ESLint, Prettier (JS), flake8, black (Python)	Industry-standard, low-overhead tools that enforce consistent style and catch common errors before tests even run.
Static Code Analysis	SonarCloud	Provides a single quality-gate view (bugs, code smells, duplication, coverage) across both JS and Python codebases.
Unit/Integration Testing	Jest (backend), pytest (ai-engine), React	Each is the de facto standard testing framework for its respective

Stage	Tool(s)	Justification
	Testing Library (frontend)	language/framework, minimizing onboarding cost.
End-to-End Testing	Docker Compose + Postman/Newman	Spins up the full stack exactly as in production and runs scripted API/user-flow checks against it.
Dependency Scanning	npm audit, pip-audit	Free, built-in tools that flag known-vulnerable dependencies before packaging.
Container Image Scanning	Trivy	Open-source scanner that checks built Docker images for OS and library vulnerabilities before they are pushed.
Secret Scanning	Gitleaks	Prevents accidental commits of API keys, database passwords, or JWT secrets into the repository.
Container Registry	GitHub Container Registry (GHCR)	Tightly integrated with GitHub Actions and the repository's existing access control, avoiding a separate registry account.
Deployment Orchestration	Docker Compose (Dev/Staging) / Kubernetes (Production)	Compose is sufficient for small Dev/Staging environments; Kubernetes gives production the rolling/blue-green updates and self-healing needed for a 24/7 public-safety system.
Secrets Management	GitHub Secrets (pipeline) + HashiCorp Vault (runtime)	GitHub Secrets protect pipeline credentials; Vault centralizes and rotates runtime secrets across environments.
Monitoring & Alerting	Prometheus + Grafana	Open-source, container-native monitoring stack well suited to tracking service health, MQTT throughput, and API latency.
Notifications	Slack Webhooks + Email (GitHub Actions notify step)	Keeps the whole team immediately aware of pipeline failures, deployments, and rollbacks.

4. Automated Testing Strategy

Testing is layered so that faster, cheaper tests run first and gate progression to slower, more expensive ones - a standard test pyramid approach integrated directly into the pipeline.

4.1 Testing Layers in the Pipeline

Layer	Scope	Tooling	Pipeline Stage
Unit tests	Individual functions: fault classification, brightness calculation, API handlers	Jest, pytest	CI - Stage 4 (every push/PR)
Integration tests	Backend ↔ database, backend ↔ AI engine, MQTT ingestion	Jest + Testcontainers / pytest + Docker	CI - Stage 4 (every push/PR)
Static/security analysis	Code smells, vulnerabilities, secrets	SonarCloud, Trivy, Gitleaks, npm audit / pip-audit	CI - Stage 5 (every push/PR)
End-to-end (E2E) tests	Full IoT-to-dashboard flow on the deployed Dev/Staging stack	Docker Compose + Postman/Newman	CD - after Dev and Staging deploys
Performance/load tests	Concurrent controller ingestion, dashboard response time	k6 or Locust	CD - Staging only, before production approval
Smoke tests	Health endpoints of all services post- deployment	curl / Newman health-check collection	CD - immediately after every deployment, incl. Production

4.2 Example GitHub Actions Test Job

```
jobs:
  test-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 18 }
      - run: npm ci
        working-directory: backend
      - run: npm run lint
        working-directory: backend
      - run: npm test -- --coverage
        working-directory: backend
```

```

- name: Enforce coverage threshold
  run: |
    COVERAGE=$(jq '.total.lines.pct'
backend/coverage/coverage-summary.json)
    if (( $(echo "$COVERAGE < 80" | bc -l) )); then
      echo "Coverage $COVERAGE% is below the 80% gate";
exit 1;
    fi

test-ai-engine:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with: { python-version: "3.11" }
    - run: |
        pip install -r ai-engine/requirements.txt
-r ai-engine/requirements-dev.txt
    - run: flake8 ai-engine/src
    - run: pytest ai-engine/tests --cov=ai-engine/src --cov-
fail-under=80

```

4.3 Quality Gate Policy

- A pull request cannot merge into develop unless: build succeeds, lint passes with zero errors, unit/integration test coverage is $\geq 80\%$, and the SonarCloud quality gate passes.
- No Docker image is pushed to the registry if Trivy reports any Critical-severity vulnerability.
- Deployment to Staging is blocked if any E2E test fails; deployment to Production is blocked if any Staging performance test breaches the defined latency/error-rate budget.

5. Deployment Strategy

Three environments provide progressively higher confidence before a change reaches live streetlights.

5.1 Environments

Environment	Purpose	Infrastructure	Trigger	Data
Development (Dev)	Continuous integration validation; shared sandbox for developers	Docker Compose on a shared dev VM/namespace	Auto-deploy on every merge to develop	Synthetic/seeded data, simulated MQTT devices
Testing/Staging	Production-like rehearsal; E2E and	Docker Compose or a small	Auto-deploy on	Anonymized copy of

Environment	Purpose	Infrastructure	Trigger	Data
	performance testing	Kubernetes namespace mirroring production topology	release/* branch creation	production data or realistic synthetic data
Production	Live system controlling real streetlights and serving real operators/citizens	Kubernetes cluster with horizontal pod autoscaling	Manual-approved deploy on merge/tag to main	Live production data

5.2 Production Deployment Strategy: Blue-Green

Production uses a blue-green deployment strategy: the new version ('green') is deployed alongside the currently live version ('blue') but receives no traffic initially. Automated smoke tests run against green; only once they pass does the load balancer/ingress cut traffic over to green. Blue is kept running for a defined window so a rollback is an instant traffic switch rather than a rebuild.

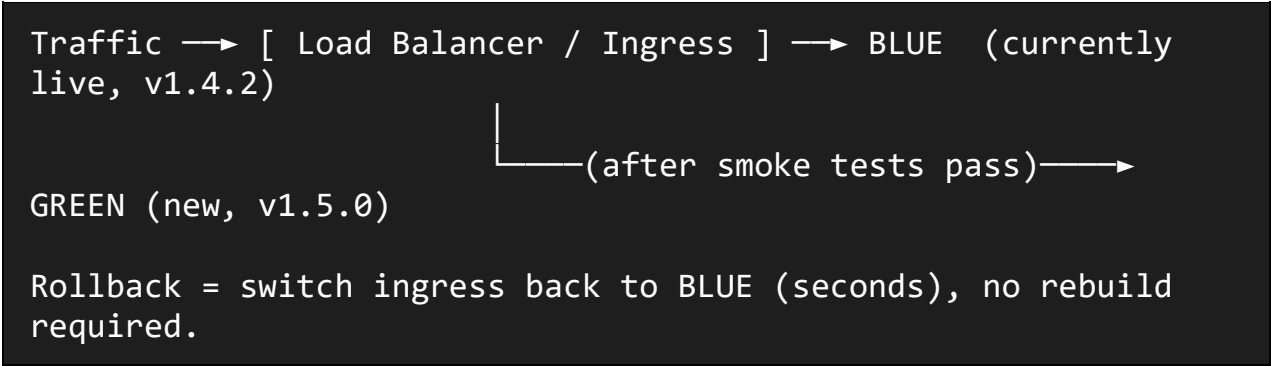


Figure 2: Blue-green production deployment

5.3 Why Blue-Green (and not pure rolling updates) for This System

- Streetlight control logic directly affects public safety - an instant, zero-downtime rollback path is more valuable here than the resource savings of a rolling update.
- Blue-green allows smoke tests to run against the new version under real infrastructure conditions before it receives any live traffic, catching issues rolling updates might expose gradually and riskily.
- For the AI engine specifically, a canary sub-strategy is layered on top: a small percentage of brightness-control decisions are routed to the new model version first, with automatic comparison against the previous model's decisions before full cutover.

6. Security, Quality Checks, Notifications, and Rollback

6.1 Security Measures in the Pipeline

Concern	Mitigation
Secrets in source control	Gitleaks scans every commit; real secrets are never committed and are injected via GitHub Secrets / Vault at runtime.
Vulnerable dependencies	npm audit and pip-audit run on every build; pipeline fails on high/critical findings.
Vulnerable container images	Trivy scans every built image before it is pushed to the registry; critical findings block the push.
Unauthorized deployment	Production deploy jobs require a protected GitHub Environment with required reviewers (manual approval gate).
Least-privilege credentials	Separate, scoped service accounts/tokens are used for Dev, Staging, and Production deployments - no shared admin credentials.
Data in transit	TLS is enforced for the API, dashboard, and MQTT broker (MQTTS) across all environments.
Audit trail	All pipeline runs, approvals, and deployments are logged in GitHub Actions history and mirrored to the application's own audit-log table (FR12 from the system requirements).

6.2 Notification Strategy

Event	Channel	Audience
CI failure (build/lint/test/scan)	Slack #ci-alerts + PR comment	Committing developer, reviewers
Successful Dev deployment	Slack #deployments	Development team
Staging deployment + E2E results	Slack #deployments + email digest	QA team, release manager
Production deployment approval request	Slack #releases + email	Release manager / approvers
Production deployment success	Slack #releases	Whole team, on-call technician lead
Automated rollback triggered	Slack #incidents + PagerDuty page	On-call engineer, engineering lead

6.3 Rollback Mechanisms

- Immutable, versioned Docker images (tagged with git SHA and semantic version) mean any previous release can be redeployed exactly as it was.
- Blue-green production deployment keeps the previous version running and reachable, so rollback is a traffic-routing change rather than a rebuild-and-redeploy cycle.
- Automated post-deployment health checks (error rate, latency, MQTT ingestion rate) monitor the new version for a defined window; if thresholds are breached, the pipeline automatically triggers rollback without waiting for a human.
- `kubectl rollout undo` (or the equivalent Compose re-deploy of the previous tag) is documented as the manual fallback if automated rollback itself needs to be re-triggered.
- Database migrations are written to be backward-compatible for at least one release, so rolling back the application does not require a simultaneous database rollback.

6.4 Example Rollback Step (GitHub Actions)

```
post-deploy-healthcheck:
  needs: deploy-production
  runs-on: ubuntu-latest
  steps:
    - name: Wait and check health endpoints
      run: |
        sleep 60
        for i in 1 2 3 4 5; do
          STATUS=$(curl -s -o /dev/null -w "%{http_code}"
https://api.streetlight.city/health)
          if [ "$STATUS" != "200" ]; then FAIL=1; fi
          sleep 30
        done
    - name: Rollback on failure
      if: env.FAIL == '1'
      run: |
        kubectl rollout undo deployment/backend -n production
        kubectl rollout undo deployment/ai-engine -n
production
    - name: Notify team
      if: always()
      uses: slackapi/slack-github-action@v1
      with:
        payload-file-path: ./ci/rollback-notification.json
```

7. Conclusion

The designed CI/CD pipeline automates the Smart AI-Based Urban Street Lighting Management System's journey from code commit to production deployment across five layered testing/quality stages, three progressively realistic environments, and a blue-green production rollout with automated health-check-driven rollback. By enforcing linting, test coverage, static analysis, and security scanning as hard gates before any code reaches develop, and by requiring manual approval before production promotion, the pipeline balances development speed with the reliability and safety expected of a system that directly controls public infrastructure. Slack/email notifications and a centralized audit trail keep the team continuously informed, while immutable versioned images and blue-green deployment ensure that any problem in production can be reversed within seconds rather than minutes or hours.